Handoff Playbook — Finish What's Left

> Written so a junior engineer (or a determined non-engineer with patience) can complete every remaining item without having to ask. If you get stuck at any step, the failure mode is described **right under** that step.

There are **six remaining work items**. Do them in this order. Each one is self-contained: you can stop after any of them and the system still works.

#	Item	Why bother	Effort
1	Fill in placeholders	The thing won't run without these	~30 min
2	Stand it up against real DB	First proof anything actually works	1–2 hr
3	Wire Ignition	Operators can finally use it	2–4 hr
4	Symphony video capture	Optional — only if you want video clips	half day
5	B2 reranker	Better retrieval quality on borderline Qs	half day
6	B13 evaluation harness	Catches regressions before they ship	1 day
7	B5 HyDE / B6 self-consistency / B11 active learning	Quality polish; defer until traffic justifies	2–3 days each

---

ITEM 1 — Fill in placeholders (30 min)

Goal: every value labeled "REPLACE_ME / change_me / PASTE_YOUR_…" is replaced with the real thing.

1A. Make a `.env` file

1. Open File Explorer to `c:\Users\jtaylo6\IgnitionChatbot\service\`.
2. Right-click → New → Text Document → name it `.env` (yes the leading dot is intentional; if Windows refuses, create it as `env.txt` then rename in PowerShell with `Rename-Item env.txt .env`).
3. Open it in any text editor. Paste:

```
SERVICE_ENV=development
API_KEY=PASTE_FROM_NEXT_STEP
DATABASE_URL=postgresql+asyncpg://chatbot:PASTE_DB_PASSWORD@localhost:5432/
ignition_chatbot
OPENAI_API_KEY=sk-PASTE_FROM_OPENAI
OPENAI_MODEL=gpt-4o-mini
OPENAI_TEMPERATURE=0.1
OPENAI_MAX_TOKENS=1500
```

```
EMBEDDING_MODEL=text-embedding-3-small
NIGHTLY_JOBS_INTERVAL_SECONDS=86400
OUTCOME_CLOSURE_ENABLED=true
```

1B. Generate the `API_KEY`

Open PowerShell and run:

```
cd c:\Users\jtaylo6\IgnitionChatbot\service
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

It will print something like `xJ3k7mP9qR2sT5vW8yA1bC4dE6fG9hJ0`. Copy that exact string and paste it into `.env` after `API_KEY=`. **Save the same string somewhere safe** — you'll paste it again into Ignition in step 3.

1C. Get / pick the `DATABASE_URL` password

- If a DBA gave you Postgres credentials → use those, replacing `chatbot` and `PASTE_DB_PASSWORD` accordingly.
- If you're standing up Postgres yourself with the included `docker-compose.yml`, you choose the password. Open `docker-compose.yml` and find `POSTGRES_PASSWORD:`. Whatever you put there is what goes in `.env`.

1D. Get the `OPENAI_API_KEY`

1. Go to https://platform.openai.com/api-keys.
2. Click "Create new secret key". Name it `ignition-coater1-prod`.
3. Copy the key (starts with `sk-`). **You can never see it again.**
4. Paste into `.env` after `OPENAI_API_KEY=`.

> If your company uses Azure OpenAI instead, leave `OPENAI_API_KEY` blank and uncomment the four `AZURE_OPENAI_*` variables in `INSTALL.md` Part 2.1, filling values from your Azure portal → Azure OpenAI resource → Keys and Endpoint.

1E. Verify

In PowerShell:

```
cd c:\Users\jtaylo6\IgnitionChatbot\service
python -c "from config.settings import get_settings; s = get_settings();
print('OK' if 'change_me' not in s.database_url and s.api_key != 'dev-key-
change-me' else 'STILL HAS PLACEHOLDERS')"
```

Expected output: `OK`. If you see `STILL HAS PLACEHOLDERS`, re-check `.env` for typos — the file has to be in `service/`, not the repo root.

ITEM 2 — Stand it up against a real Postgres (1–2 hr)

Goal: the FastAPI service boots, connects to Postgres, accepts a `/api/chat` POST, and returns a grounded answer.

2A. Start Postgres

```
cd c:\Users\jtaylo6\IgnitionChatbot
docker compose up -d postgres
```

Wait ~30 sec, then:

```
docker compose logs postgres --tail 20
```

You should see `database system is ready to accept connections`. If you see "port already in use" → something else is on 5432. Edit `docker-compose.yml`, change `"5432:5432"` to `"5433:5432"`, and update `.env` `DATABASE_URL` to `:5433`.

2B. Run the schema scripts

```
docker compose exec -T postgres psql -U chatbot -d ignition_chatbot < scripts/
setup_database.sql
docker compose exec -T postgres psql -U chatbot -d ignition_chatbot < scripts/
seed_reference_data.sql
```

Expected: each command prints a long list of `CREATE TABLE`, `CREATE INDEX`, `INSERT 0 N`, with **no `ERROR:`** lines.

If you see `ERROR: extension "vector" is not available`: the Postgres image doesn't have pgvector. Open `docker-compose.yml`, change the image to `pgvector/pgvector:pg16`, then `docker compose down -v && docker compose up -d postgres` and retry.

2C. Seed line memory + facts

```
cd service
pip install -r requirements.txt
python -m scripts.seed_initial_data
```

Expected last line: `Seeded N memory entries`.

2D. Boot the service

```
uvicorn main:app --reload --port 8000
```

Expected log lines (in JSON):

- `service_starting`
- `db_connected`
- `embeddings_provider_ready`
- `Application startup complete`

If `embeddings_provider_ready` does not appear, your `OPENAI_API_KEY` is wrong or you're offline. The service still boots — you just can't get real answers.

2E. Smoke test

In a new PowerShell window:

```
$key = (Get-Content c:\Users\jtaylo6\IgnitionChatbot\service\.env | Select-String "^API_KEY=").ToString().Split("=")[1]
curl -Method GET http://localhost:8000/api/health -Headers @{ "X-API-Key" = $key }
```

Expected JSON: `{"status":"ok","db":"ok","embeddings":"ok"}`.

ITEM 3 — Wire Ignition (2–4 hr)

Goal: an operator opens the Perspective ChatView and gets real answers.

Follow `INSTALL.md` Part 5 verbatim. The two non-obvious bits:

3A. Tag path placeholder fix

Open `ignition/perspective/gateway_wiring.py`. Find the line near 165:

```
line_id = tag.tagPath.split("/")[1]
```

Open Designer → Tag Browser → expand to find a Coater 1 alarm tag. Right-click → Copy → Tag Path. Paste it somewhere visible. It will look like `[UnifiedNamespace]Shaw/F0004/Coating/Coater1/Alarms/HighTemp`.

Count how many slashes precede `Coater1`. In the example: positions 0=`[UnifiedNamespace]Shaw`, 1=`F0004`, 2=`Coating`, 3=`Coater1`. So the fix is:

```
line_id = tag.tagPath.split("/")[3]
```

Edit the file accordingly. **Do not skip this** or every alarm will be attributed to `"F0004"` instead of `"coater1"`.

3B. The `ai.config` script

In Designer → Project Browser → Scripting → right-click → New Package → name it `ai`. Inside `ai`, create a Script named `config`. Paste the block from `INSTALL.md` Part 5, then replace each `<YOUR_…>` exactly as described in `GAP_ANALYSIS.md` §3.4.

3C. Verify pairing

Open the Perspective ChatView (the one specified in `ignition/perspective/CHAT_VIEW_SPEC.md`), type `"what's the current state of zone 3?"`, press Send.

Expected: a paragraph answer with `[1]`, `[2]` style citations and a `CONFIDENCE:` line at the end.

If you get a red error toast `"AI service unreachable"`:

- From the Gateway machine, run `curl http://<HOST_IP_OF_AI_SERVER>:8000/api/health`. If that fails, it's a firewall (open port 8000 inbound on the AI host).
- If `curl` works but the view fails, the API key in `ai.config` doesn't match the one in the service's `.env`. Re-paste.

ITEM 4 — Symphony video capture (half day, OPTIONAL)

You only need this if your operators want a video clip auto-attached when a defect event is anchored.

1. Open `service/services/symphony_capture.py`.
2. Find the function `capture_clip(camera_id, start, end)`.
3. Replace the body (currently returns a `extraction_status: "stub"` dict) with a real call to your Symphony deployment. Pseudocode:

```
import httpx
from config.settings import get_settings
async def capture_clip(camera_id, start, end):
    s = get_settings()
    async with httpx.AsyncClient(timeout=30) as cx:
        r = await cx.post(
            f"{s.symphony_base_url}/api/clips",
            headers={"Authorization": f"Bearer {s.symphony_api_key}"},
            json={"camera": camera_id, "start": start.isoformat(),
                  "end": end.isoformat(), "format": "mp4"},
        )
    r.raise_for_status()
    body = r.json()
    return {
        "storage_handle": body["clipUrl"],
        "extraction_status": "complete",
        "camera_location": body.get("location", camera_id),
    }
```

1. Add `symphony_base_url` and `symphony_api_key` to `Settings` in `service/config/settings.py` (mirror the existing pattern).
2. Add the two values to `.env`. Get them from your Symphony admin console → API tab.

ITEM 5 — B2 cross-encoder reranker (half day)

The full implementation hint is already inside `service/services/reranker.py` as comments. Steps:

1. **Open `service/requirements.txt`**, find the line that says `# sentence-transformers==X.Y.Z # B2 reranker`, **uncomment it**.
2. `pip install -r requirements.txt`. This will download ~500 MB of torch — be patient.

3. **Open `service/services/reranker.py`**. Replace the body of `_load_model()` with:

```
def _load_model():
    global _MODEL
    if _MODEL is None:
        from sentence_transformers import CrossEncoder
        _MODEL = CrossEncoder("BAAI/bge-reranker-base", max_length=512)
    return _MODEL
```

1. Replace the body of `rerank(...)` (currently `return candidates[:top_k]`) with:

```
async def rerank(query, candidates, top_k):
    if not candidates:
        return []
    model = _load_model()
    pairs = [(query, c.chunk_text) for c in candidates]
    import asyncio
    scores = await asyncio.get_running_loop().run_in_executor(
        None, model.predict, pairs)
    scored = sorted(zip(candidates, scores), key=lambda x: x[1], reverse=True)
    out = []
    for c, s in scored[:top_k]:
        c.rerank_score = float(s)
        out.append(c)
    return out
```

1. **Open `service/services/retrieval.py`**, find `retrieve_chunks_hybrid`. After the MMR step (look for `mmr_select`), before the final `return candidates[:top_k]`, add:

```
from services.reranker import rerank
candidates = await rerank(query, candidates, top_k=top_k)
```

1. Add `reranker_enabled: bool = True` to `Settings`. Wrap the call in step 5 with `if get_settings().reranker_enabled:` so you can flip it off without redeploying.
2. Run the test suite: `pytest -q`. The first run will be slow (model download) but should pass. If a test fails because it asserts on a specific candidate ordering, the test is now wrong — update the expected order or `monkeypatch` the reranker to passthrough.

ITEM 6 — B13 evaluation harness (1 day)

Open `service/eval/harness.py`. Three functions to fill in.

6A. `load_cases(yaml_path)`

```
import yaml
from pathlib import Path
def load_cases(yaml_path):
    raw = yaml.safe_load(Path(yaml_path).read_text())
```

```
    return [EvalCase(**c) for c in raw["cases"]]
```

You also need to write the actual YAML cases file. Start with 10 cases. Format (save as `service/eval/golden_cases.yaml`):

```
cases:
  - id: zone3_drift_2026_03_14
    query: "Why did we get coating weight variation on Friday's first run?"
    curated_context_fixture: fixtures/zone3_drift.json
    expected_failure_mode: coating_weight_var
    expected_citation_ids_at_least: ["chunk:wo_4521", "event:drift_2026_03_14"]
    expected_confidence_one_of: [likely, confirmed]
```

Each case needs a fixture JSON — capture one by hitting `/api/chat` with verbose audit on, then copy the `request_payload` from `audit_log` into `fixtures/<id>.json`.

6B. `run_eval(cases, base_url, api_key)`

```
import httpx
async def run_eval(cases, *, base_url, api_key):
    results = []
    async with httpx.AsyncClient(timeout=60) as cx:
        for c in cases:
            payload = json.loads(Path(c.curated_context_fixture).read_text())
            payload["query"] = c.query
            r = await cx.post(f"{base_url}/api/chat",
                              json=payload,
                              headers={"X-API-Key": api_key})
            r.raise_for_status()
            results.append(score(c, r.json()))
    return results
```

Plus a `score(case, response) -> EvalResult` helper that checks:

- every `expected_citation_ids_at_least` ID appears in `response["sources"]` → boolean per case → recall
- `response["sources"]` IDs ⊆ everything that was retrieved → precision
- `response["confidence"]` ∈ `case.expected_confidence_one_of`
- if `case.expected_failure_mode`, compare to `response["failure_mode_classification"]`

6C. `summarize(results)`

```
def summarize(results):
    n = len(results)
    return {
        "n_cases": n,
        "citation_recall": sum(r.citation_recall for r in results) / n,
        "citation_precision": sum(r.citation_precision for r in results) / n,
        "confidence_honored_pct": sum(r.confidence_ok for r in results) / n,
        "fm_accuracy": sum(r.fm_correct for r in results) / sum(1 for r in
```

```
results if r.fm_expected),
    }
```

6D. CLI

Add to bottom of `harness.py`:

```
if __name__ == "__main__":
    import argparse, asyncio, json
    p = argparse.ArgumentParser()
    p.add_argument("--cases", default="service/eval/golden_cases.yaml")
    p.add_argument("--base-url", default="http://localhost:8000")
    p.add_argument("--api-key", required=True)
    args = p.parse_args()
    cases = load_cases(args.cases)
    results = asyncio.run(run_eval(cases, base_url=args.base_url,
api_key=args.api_key))
    print(json.dumps(summarize(results), indent=2))
```

Run with:

```
python -m eval.harness --api-key $env:API_KEY
```

Target benchmarks (set these as your "do not regress" line):

- `citation_recall ≥ 0.85`
- `citation_precision ≥ 0.75`
- `confidence_honored_pct ≥ 0.90`
- `fm_accuracy ≥ 0.80`

ITEM 7 — Quality-polish items (defer until justified)

Don't build B5 HyDE, B6 self-consistency, or B11 active-learning *proactively*. Build them **only after** the eval harness (item 6) shows a measurable problem they'd solve:

If eval shows…	Then build…
Recall < 0.7 on rare-terminology queries	B5 HyDE
Confidence/answer flips between repeat runs	B6 self-consistency (k=3 vote)
Same wrong chunks keep getting cited despite corrections	B11 active-learning trainer

For each, open the corresponding stub file in `service/services/` — they all have implementation skeletons in their docstrings.

Done-checklist

- [ ] §1 done — `.env` filled, `API_KEY` saved somewhere
- [ ] §2 done — `/api/health` returns `{"db":"ok","embeddings":"ok"}`
- [ ] §3 done — Perspective ChatView returns a real answer with citations
- [ ] §4 (optional) — Symphony clip URL appears on a defect anchor
- [ ] §5 (when ready) — `pytest -q` passes with reranker on
- [ ] §6 (when ready) — `python -m eval.harness` prints all four metrics

Once §1–§3 are done you have a working system. Everything after is optimisation.